Planning at the Speed of Control

Profiling and fusing world-model robot planning into a single GPU kernel

Antoine Nguyen
Student number 24433599

Project report
[Faculty / School], University of Technology Sydney

Supervisor: [Supervisor name]
October 2026

Executive summary

Robots that plan with a learned world model imagine the outcome of hundreds of candidate actions before they move, and that imagination takes time. This project studies LeWorldModel (LeWM), a compact AI world model that plans by searching over action sequences in its own learned representation, and asks why one plan for one robot still takes 622 milliseconds on an NVIDIA H100, one of the fastest GPUs available, and how much of that time can be removed without retraining the model or making the robot less successful. We treat the question as a data analytics problem. We collected per-call latency records, planner telemetry and component timings, prepared them into tidy datasets, and used them to build a stage-by-stage compute profile of the planner. The profile showed three things. A third of the time (207 ms) is spent re-encoding the same two camera images 60 times. Most of the rest (368 ms) is the planner's 30 optimisation iterations, each of which costs the same 20 ms in PyTorch whether it evaluates 60 candidate plans or 300: a fitted latency model attributes 99% of the cost to fixed per-launch overhead rather than to computation. And the arithmetic itself is negligible, because the model is tiny and the work is a long chain of small dependent steps. We designed the solution against that profile. It encodes each image once, uses a sample-efficient optimiser (iCEM) that evaluates 3.2 times fewer candidates, and runs the whole plan, from the raw camera frame to the chosen action distribution, as one persistent GPU program (a "megakernel") written with the ThunderKittens library. One plan now takes 46.5 ms (13.4 times faster; 95% CI 13.0 to 13.5), and 3.9 times faster than a carefully optimised PyTorch implementation, with a 90th percentile of 47.0 ms. Task success on 300 paired Push-T episodes is unchanged: 260 against 261 successes, a paired difference of −0.3 percentage points (95% CI −4.3 to +3.7; exact McNemar p = 1.00). The kernel's numerical error matches the accepted half-precision path. The solution pack contains the datasets and a single Python program that regenerates every number and chart in this report.

Acknowledgements

I thank my supervisor for their guidance, the authors of LeWorldModel and stable-worldmodel for releasing the model, the planner and the benchmark harness, and the Hazy Research group for ThunderKittens. GPU time was rented on Modal.

Declaration

The assignment cover sheet is attached to this submission. I declare that this report is my own work, that it has not been submitted for any other degree or diploma, and that all sources of information and assistance have been acknowledged. Generative AI tools were used as a coding assistant, as a literature search aid and as a writing aid; every source cited was checked against the primary document, and every result reported was produced by the code and data in the solution pack (Appendix B). No ethics approval was required: the research involved no human or animal participants.

Table of Contents

Executive summary 2
Acknowledgements 2
Declaration 2
1. Introduction 5
1.1 Motivation 5
1.2 Aim and objectives 5
1.3 Relevance to data analytics and AI 5
1.4 Scope and structure 6
2. Background and literature review 7
2.1 World models that plan in a learned representation 7
2.2 Sampling-based planning: CEM and iCEM 7
2.3 Running small AI models fast on a GPU 8
2.4 The gap 8
3. Methodology 9
3.1 Research design 9
3.2 System under study 9
3.3 Data collection 10
3.4 Data preparation and preprocessing 11
3.5 Analysis methods 11
3.6 Evaluation design 12
3.7 Tools and implementation 12
4. Compute-profile analysis 13
4.1 Where a plan's time goes 13
4.2 Finding 1: a third of the time re-encodes the same two images 13
4.3 Finding 2: the search is bound by overhead, not computation 13
4.4 Finding 3: sample-efficient search only pays once overhead is gone 14
4.5 From findings to design 14
5. Solution design and development 16
5.1 Encode each image once 16
5.2 Search with iCEM 16
5.3 One GPU program per plan 16
5.4 The encoder inside the same program 18
5.5 Numerical design 18
5.6 Deployment 19
6. Evaluation 20
6.1 Latency 20
6.2 The new compute profile 20
6.3 Task success 21
6.4 Numerical fidelity 22
6.5 Validating the scaling model 22
7. Discussion 24
7.1 Were the objectives met? 24
7.2 Why the solution works 24
7.3 Position relative to other approaches 24

7.4 Limitations....25
8. Conclusion and future work....26
8.1 Conclusion....26
8.2 Future work....26
References....27
Appendix A. Data dictionary....32
Appendix B. Solution pack....33
Appendix C. Configuration and repeatability....34
Appendix D. Compute and cost....35
Appendix E. Code and data availability....36

1. Introduction

1.1 Motivation

A world model is an AI model that predicts what will happen next. Given a camera image of a scene and a proposed action, it predicts the next state of the scene, and by chaining predictions it can imagine the consequences of a whole sequence of actions. A robot that owns such a model can plan: it imagines many candidate action sequences, scores each by how close its imagined outcome comes to a goal, and executes the best. This is how a growing family of robot-learning systems works, and the family built on the joint-embedding predictive architecture (JEPA) is among the most promising, because it learns to predict in a compact learned representation rather than in pixels (Assran et al., 2023; Zhou et al., 2025; Assran et al., 2025).

Planning by imagination is expensive. Every plan evaluates thousands of imagined futures, and for the large world models of the field a single plan has taken from 16 seconds to nearly a minute (Zhou et al., 2025; Assran et al., 2025). LeWorldModel (LeWM) was designed to fix that. It is a 15-million-parameter JEPA, small enough to train on one GPU in a few hours, that encodes each image into a single 192-number vector and plans up to 48 times faster than its predecessors, "in under one second" (Maes, Le Lidec, Scieur, et al., 2026).

Under one second is fast for a world model and slow for a robot. A controller that replans as it moves must finish each plan within its control period; a robot arm controlled ten times per second has 100 milliseconds for sensing, planning and acting together. Measured on one H100 GPU, LeWM's released planner takes 622 ms per plan. The model is so small that the arithmetic of a plan should take a few milliseconds, so the remaining time must be going somewhere else. Finding out where, and removing it, is the subject of this report.

1.2 Aim and objectives

The aim is to reduce the latency of one LeWM plan for one robot on one GPU to the time scale of control, without retraining the model and without reducing task success. Four objectives make the aim measurable.

- **O1. Profile.** Collect and analyse latency data to build a stage-by-stage compute profile of the reference planner and identify what limits its speed.
- **O2. Design and implement.** Design an inference pipeline that removes the bottlenecks found in O1, keeping the trained model's weights and the planner's search budget (horizon, iterations, elite count) unchanged.
- **O3. Evaluate.** Measure the new pipeline's end-to-end latency and closed-loop task success against the reference and against a strong optimised PyTorch implementation on the same hardware, with paired statistical tests.
- **O4. Deploy.** Package the solution as a planner that plugs into the existing evaluation harness, and the analysis as a single program that regenerates every result from the collected data.

1.3 Relevance to data analytics and AI

The project sits where AI systems meet data analytics. The object of study is an AI system: a self-supervised world model and the search algorithm that uses it to make decisions. The method is an analytics pipeline applied to that system's behaviour. Its data are

measurements: 11,942 timed policy calls, 900 episode outcomes, planner telemetry and component micro-benchmarks. The pipeline follows the standard steps of an analytics project, from data collection, preparation and preprocessing, through pattern discovery and modelling, to evaluation and deployment (Section 3.1). The patterns it discovers drive the engineering. A fitted latency model distinguishes work that scales with the number of candidate plans from fixed overhead that does not, and that distinction decides which optimisations can pay. The evaluation is statistical, with bootstrap confidence intervals, paired tests and a power analysis, because a faster AI system is only useful if it is shown to behave the same. And the deployed artefact is an AI inference engine, the part of an AI system that runs in production.

1.4 Scope and structure

The study covers one model (LeWM's released Push-T checkpoint), one task (Push-T, pushing a T-shaped block into a target pose), one GPU type (NVIDIA H100) and one robot per GPU, which is the deployment case. The model is used as released; no training is involved. Section 2 reviews the background. Section 3 describes the methodology. Section 4 reports the compute-profile analysis, Section 5 the solution designed from it, and Section 6 its evaluation. Section 7 discusses the results and their limitations, and Section 8 concludes with future work. The appendices give the data dictionary, the solution pack, the configuration and the compute cost.

2. Background and literature review

2.1 World models that plan in a learned representation

The joint-embedding predictive architecture learns two networks: an encoder that maps an observation to a compact vector (an embedding) and a predictor that predicts the embedding of a future observation from the embedding of the present one. Because the prediction target is an embedding rather than a picture, the model can ignore details that do not matter for what happens next (Assran et al., 2023; Bardes et al., 2024). Conditioning the predictor on actions turns it into a dynamics model, and a goal image becomes a target in embedding space. DINO-WM showed that such a model, built on frozen DINOv2 features, can plan manipulation and navigation tasks zero-shot (Zhou et al., 2025; Oquab et al., 2024); PLDM made the case for planning with latent dynamics learned from reward-free data (Sobal et al., 2025); and V-JEPA 2-AC planned with real robot arms after pre-training on video (Assran et al., 2025). A systematic study found the cross-entropy method the best planner for these models on Push-T (Terver et al., 2026).

LeWM is the system studied here (Maes, Le Lidec, Scieur, et al., 2026). Its encoder is a small Vision Transformer (ViT-tiny; Dosovitskiy et al., 2021) and its predictor a 6-layer transformer conditioned on actions through adaptive layer normalisation (AdaLN; Peebles & Xie, 2023). It is trained end to end from pixels with a regulariser, SIGReg, that keeps the embeddings well spread (Balestriero & LeCun, 2025), and it is evaluated with the stable-worldmodel platform (Maes, Le Lidec, Facury, et al., 2026). Its successors make planning cheaper by changing the model: Fast LeWorldModel retrains the predictor to roll out a candidate in one pass instead of five, halving planning time (Gao & Xu, 2026), and an inverse-dynamics policy trained on top of LeWM replaces search altogether (Nguyen et al., 2026). Planning time in this literature is reported coarsely, as seconds per plan, and is always reduced by changing the model or the algorithm, never by examining how the planner is executed.

2.2 Sampling-based planning: CEM and iCEM

The cross-entropy method (CEM) searches by sampling (Rubinstein, 1999; de Boer et al., 2005). It draws many candidate action sequences from a Gaussian distribution, scores each, keeps the best (the elites), refits the Gaussian to the elites and repeats. With learned models it became the standard planner of model-based reinforcement learning (Chua et al., 2018; Hafner et al., 2019; Hansen et al., 2022). LeWM uses it with 300 candidates, 30 elites and 30 iterations per plan, so a plan evaluates 9,000 imagined futures.

The improved cross-entropy method, iCEM, makes CEM sample-efficient (Pinneri et al., 2021). Its most important features for this project are a population that shrinks over the iterations as the search narrows, and the reuse of the best candidates of one iteration in the next. On its benchmarks it needed 2.7 to 22 times fewer samples for the same result. A caution comes from related work: a CEM variant that cut samples a hundredfold by adding gradient steps was only 25% faster than the CEM it replaced, because cutting the number of samples does not cut time unless the cost of each iteration falls too (Huang et al., 2021). Whether iCEM saves time therefore depends on how the planner is executed, which is one of the questions this project answers with data.

2.3 Running small AI models fast on a GPU

A GPU runs work as kernels: programs launched by the CPU, each doing one operation (a matrix multiplication, a normalisation) across thousands of parallel threads. Frameworks such as PyTorch launch one or more kernels per operation. For large models this is efficient, because each kernel has enough work to keep the GPU busy. For small models at batch size one it is not: each kernel finishes in a few microseconds, and the time between kernels (the launch itself, the Python code that issues it, and the wait for results) dominates. The roofline model describes performance limited by arithmetic or by memory bandwidth (S. Williams et al., 2009); analyses of small-batch transformer inference show that neither limit applies when latency and data movement dominate (Pope et al., 2023; Ivanov et al., 2021).

Three remedies exist, in increasing order of ambition. CUDA Graphs record a sequence of launches and replay it with less CPU overhead (Gray, 2019). Compilers such as PyTorch 2's `torch.compile` fuse neighbouring operations into fewer, larger kernels (Ansel et al., 2024; Tillet et al., 2019); FlashAttention fuses a whole attention computation into one kernel that keeps its intermediate results on chip (Dao et al., 2022; Shah et al., 2024). The most ambitious remedy is the persistent kernel or megakernel: one long-running GPU program that performs a whole computation, so that there are no launches and no gaps at all (Gupta et al., 2012; Diamos et al., 2016). Megakernels returned in 2025 for language-model inference, where one kernel running a whole Llama forward pass cut latency below a millisecond on an H100 (Spector, Juravsky, Sul, Dugan, et al., 2025; Nrusimha et al., 2025), and compilers that build megakernels automatically followed (Cheng et al., 2026). ThunderKittens, the library used here, exposes the H100's tensor-core instructions and asynchronous memory copies through simple tile abstractions (Spector, Arora, et al., 2025). The H100's thread-block clusters and distributed shared memory, which let neighbouring processors exchange data without going through main memory, are what make a kernel that spreads one small model across many processors practical (Andersch et al., 2022; Luo et al., 2024).

2.4 The gap

In robotics, fast sampling-based planning on GPUs is well established with hand-written physics models: MPPI controllers and motion planners reach millisecond rates through custom kernels (G. Williams, Aldrich, & Theodorou, 2017; Bhardwaj et al., 2022; Sundaralingam et al., 2023; Vlahov et al., 2024). For learned world models the field has improved planning speed only by changing the model or the planner. No work has profiled where a world-model planner's time goes when it is executed, measured whether sample-efficient planning saves time in practice, or fused a whole planning loop, with a neural encoder, a transformer rollout, scoring, elite selection and distribution updates over every iteration, into one GPU program. This project does all three, for LeWM.

3. Methodology

3.1 Research design

The research is quantitative and follows an analytics pipeline (Figure 1). Data about the planner's behaviour are collected with timing instruments, prepared into tidy tables, and analysed to discover where the time goes and why. The patterns found determine the design of the solution, and the solution is evaluated with the same instruments and statistics, then packaged for deployment. Any design change that does not improve the measured profile without changing behaviour is discarded.

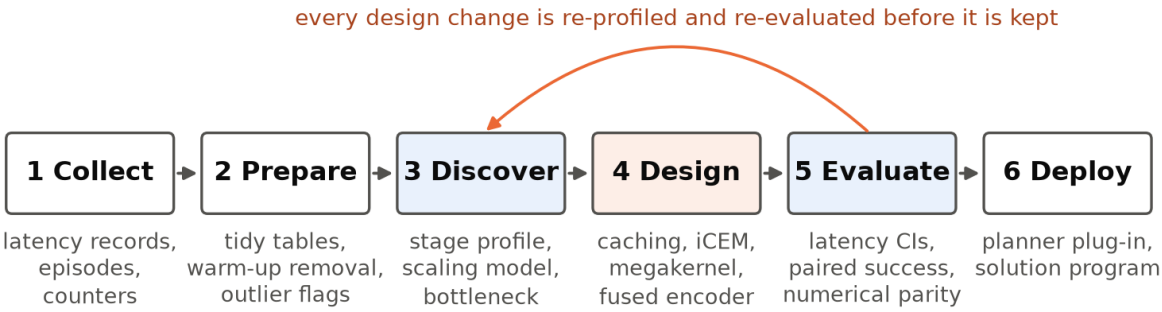


Figure 1. The analytics pipeline of the project. Steps 1 to 3 produce the compute profile (Section 4); step 4 is the solution (Section 5); step 5 its evaluation (Section 6); step 6 the deliverables (Appendix B). The orange arrow is the loop by which every design change was re-profiled before it was kept.

Three design principles were fixed at the outset, and each answers a specific threat to validity.

- **Measure what a deployment would feel.** The primary metric is the end-to-end latency of one planning call for one robot, from camera image in to action out, including image preprocessing, encoding, the search and the copy back to the CPU. A kernel timed in isolation can hide costs that the robot still pays, so isolated timings are used only to explain the end-to-end number.
- **Equal hardware, strong baseline.** Every configuration runs on the same GPU model in the same software container. The new pipeline is compared with the released reference planner and with the best PyTorch implementation available, which already uses the standard remedies of Section 2.3 (compilation, CUDA Graph replay, TF32 and BF16 arithmetic). Beating only an unoptimised reference would not show that the design matters.
- **Behaviour is measured in the loop, on paired episodes.** A faster planner that makes the robot fail more often is worse, and small numerical changes can alter which candidate wins a close race. Every configuration is evaluated on the same 300 Push-T episodes, whose start states are fixed by the random seed, so differences are measured episode by episode.

3.2 System under study

Table 1 lists the facts of the model and planner that the analysis depends on, taken from the released code and checkpoint. Figure 2 shows how one plan is computed.

Table 1. The reference system: LeWM's released Push-T checkpoint and planner.

item	value
encoder	ViT-tiny: 12 layers, width 192, 3 attention heads, 14 × 14 patches of a 224 × 224 image (257 tokens), projected to a 192-number embedding
predictor	6-layer transformer, width 192, 16 heads, MLP width 2,048, conditioned on actions by AdaLN; looks back over up to 3 past steps
action	a block of 5 two-dimensional pushes (10 numbers) per planning step; plans cover 5 steps
planner	CEM: 300 candidates, top 30 elites, 30 iterations (9,000 imagined rollouts per plan)
cost	squared distance between the predicted final embedding and the goal image's embedding
control loop	a plan is computed at policy calls 1 and 26 of a 50-call episode; the calls in between execute the planned actions
arithmetic	32-bit floating point (FP32), PyTorch operations launched one at a time ("eager" mode)

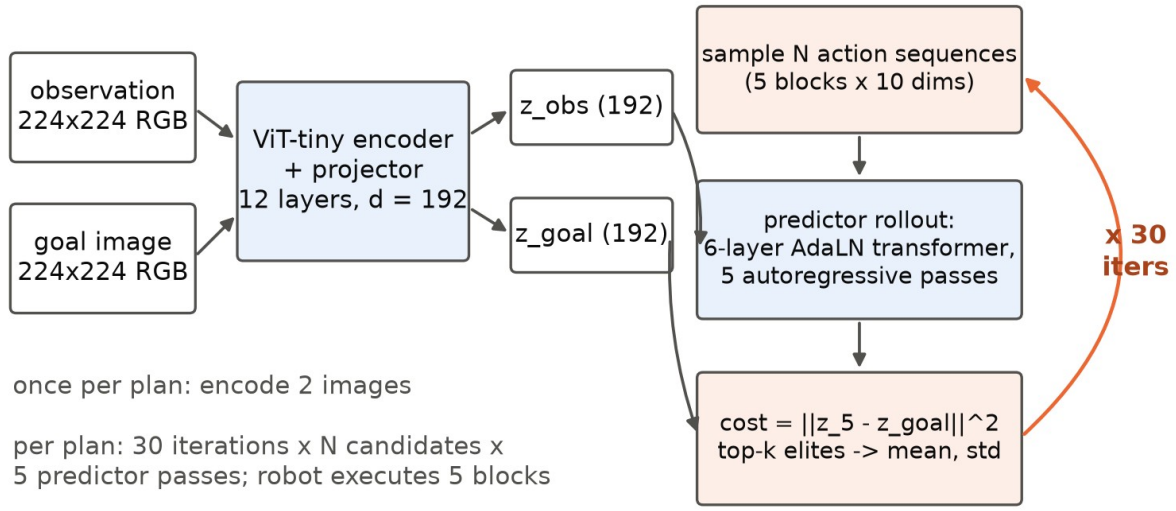


Figure 2. How LeWM plans. Both images are encoded to embeddings; the planner samples candidate action sequences, rolls each through the predictor for five steps, scores the final embedding against the goal, refits its sampling distribution to the best candidates and repeats 30 times.

Two properties of this system shape the problem. The model is small, about 15 million parameters, so scoring 300 candidates is under 100 billion floating-point operations, less than a tenth of a millisecond of the H100's tensor-core capacity, and all its weights fit in the GPU's 50 MB on-chip cache. And the computation is deep: one plan is 30 sequential iterations of a 5-step rollout through 6 layers, a chain of 900 transformer-layer evaluations, each of which depends on the one before.

3.3 Data collection

Four instruments produced the data (Table 2). All ran on NVIDIA H100 80GB HBM3 GPUs rented from Modal, with PyTorch 2.11.0 and CUDA 12.8 in one container image.

Table 2. Datasets collected. All are in the solution pack's data folder; Appendix A gives the columns.

dataset	rows	instrument	content
calls.csv	11,942	CUDA event timers around every policy call, inside the benchmark harness	latency of every call of 61 one-robot episodes for each of three systems and four probe configurations
episodes.csv	900	Push-T evaluation, 3 seeds × 100 episodes per system	success (0 or 1) of every episode
cost_scaling.csv	15	micro-benchmark of one scoring call at 60 to 300 candidates	median latency per execution method
components.csv	11	component micro-benchmarks and planner telemetry	encoder and search timings, synchronisation costs, encoder calls per plan
parity.csv	15	comparison with FP32 on identical inputs	maximum relative cost error and agreement of the top-30 candidates

The latency episodes use seeds 1000 to 1060, one robot per process, so nothing is shared between robots. The seven configurations in calls.csv are the three **systems** compared in the evaluation (the reference planner, the optimised PyTorch planner and the new pipeline) and four **probes**: configurations that differ from a system in exactly one respect, run only to isolate an effect for the profile. Two probes encode the images once per plan, with CEM and with iCEM. The other two run the new kernel with CEM and with iCEM, using the PyTorch encoder.

3.4 Data preparation and preprocessing

The raw records are line-delimited JSON files written by the harness, one per run. The program's `prepare` step extracts them into the five tidy CSV tables, one row per observation. The `analyse` step then applies three preprocessing rules before any statistic is computed.

- **Warm-up removal.** The first seed of every configuration is discarded, because the first calls of a fresh process include one-off costs (memory allocation, kernel loading, and for the compiled baseline the compilation itself). This leaves 72 to 76 planning calls per configuration.
- **Call classification.** Policy calls 1 and 26 of each episode are planning calls; all others only pop an already-planned action and are kept as a separate step-call distribution, which measures the fixed cost of a policy call.
- **Outlier flagging.** Planning calls outside 1.5 interquartile ranges of their configuration's quartiles are counted and reported but kept. Medians are used as the location statistic because the latency distributions are skewed.

3.5 Analysis methods

Stage decomposition. A plan's time is divided into stages: CPU preprocessing, image encoding, the search (rollouts and distribution updates), and synchronisation with the CPU. Each stage is estimated from a direct measurement where one exists (the encoder's time from its micro-benchmark, multiplied by the number of encoder calls counted in the planner's telemetry; the search from the probe that removes the redundant encoding), and the remainder is attributed by difference and labelled as such.

Scaling model. To tell computation apart from overhead, the latency of one scoring call is modelled as a straight line in the number of candidates N, latency = a + b·N, fitted by least squares for each execution method. The intercept a is the fixed cost of a call, paid whatever its size; the slope b is the cost of each additional candidate. The share a / (a + 300b) is the fraction of a full-size call that is fixed overhead. A share near 1 identifies a launch- or latency-bound regime, in which reducing the number of candidates cannot help. The fitted model also predicts what iCEM should save: with 30 calls in both schedules, the predicted speed-up is (30a + 9,000b) / (30a + 2,820b).

Per-layer cost. Dividing the search time by the 900 layer evaluations of a plan gives the average cost of one layer evaluation, which can be compared with its arithmetic content and with the measured cost of the GPU's synchronisation primitives.

3.6 Evaluation design

Table 3 summarises the evaluation. Latency is summarised by the median with a bootstrap 95% confidence interval (10,000 resamples over plans) and by the 90th percentile, which matters for a controller with a deadline. Speed-ups are ratios of medians with bootstrap intervals. Task success is reported with Wilson 95% intervals (Wilson, 1927). Systems are compared episode by episode: the counts of episodes won by each side, an exact McNemar test (McNemar, 1947) and a paired bootstrap interval on the difference in success rate (Efron & Tibshirani, 1993). Because the claim is that behaviour does not change, the evaluation's resolution is stated: with the observed 13% of episodes differing between systems, 300 pairs detect a difference of about 6 percentage points at 80% power, and a 2-point difference would need about 2,500 pairs (Connor, 1987). Before any closed-loop run, the new pipeline's scoring was compared with FP32 on identical candidates, and accepted only if its error was no larger than that of the optimised PyTorch planner's BF16 arithmetic, whose closed-loop behaviour had already been accepted.

Table 3. Evaluation protocol.

item	setting
systems	reference planner; optimised PyTorch planner (iCEM, compiled, TF32/BF16); new pipeline (LeWM-MK)
latency	61 one-robot episodes per system (first dropped), 72 to 76 plans; median, bootstrap 95% CI, 90th percentile
task success	300 Push-T episodes (seeds 0 to 2 × 100), identical for every system; Wilson intervals; paired McNemar and bootstrap
numerical fidelity	8 scoring cases against FP32: maximum relative error and top-30 agreement; encoder embedding error
hardware and software	one NVIDIA H100 80GB HBM3; PyTorch 2.11.0, CUDA 12.8; ThunderKittens commit 67 845f5f

3.7 Tools and implementation

The planner and harness are the released LeWM and stable-worldmodel code with an extended benchmark harness. The new kernel is one CUDA source file (megakernel/lewm_mk.cu, about 1,600 lines) with a Python packer, runtime and planner class. GPU work ran on Modal. The analysis is one Python program (solution_pack/lewm_flash.py) that needs only numpy and matplotlib; it regenerates every table, statistic and data figure in this report from the CSV datasets in about five seconds on a laptop.

4. Compute-profile analysis

This section reports what the data showed about the reference planner. Its findings fixed the design of Section 5.

4.1 Where a plan's time goes

The reference planner's median planning call takes 622.4 ms (95% CI 605.5 to 626.7) and is spread widely, from 556 to 759 ms. Planner telemetry from the 300-episode evaluation shows that 99.3% of a planning call is spent inside the search itself, so the policy code around it is not the problem. Table 4 breaks the call into stages.

Table 4. Compute profile of one reference planning call (median, one robot, one H100).

stage	ms	share	how it was measured
CPU preprocessing and policy code	2.1	0.3%	median of the calls that only pop a planned action
image encoding	207.4	33.3%	60 encoder calls per plan (planner telemetry) × 3.46 ms per image (micro-benchmark: 6.91 ms for observation and goal)
predictor rollouts and distribution updates	368.1	59.1%	probe that encodes once per plan: its planning call minus its policy cost minus one encoding
CPU synchronisation and other	44.8	7.2%	remainder, by difference
total	622.4	100%	end-to-end median

Three findings follow from the profile and from the measurements behind it.

4.2 Finding 1: a third of the time re-encodes the same two images

A plan has two distinct images, the current observation and the goal, but the planner's telemetry counts 60 encoder calls per plan: the released planner treats the encoder as part of the scoring function and re-encodes both images in each of its 30 iterations. The images do not change during a plan, so 58 of the 60 calls are redundant, and they cost about 207 ms, a third of the plan. This is pure waste: removing it changes no computed value.

4.3 Finding 2: the search is bound by overhead, not computation

The search takes 368 ms for 30 iterations, 12.3 ms per iteration, or 409 µs per transformer-layer evaluation. The arithmetic of one layer evaluation for 300 candidates is a few billion floating-point operations: about 3 µs of the H100's BF16 tensor-core capacity, and still only tens of microseconds even at its FP32 rate without tensor cores. More than nine-tenths of the search time is therefore not computation.

The scaling model identifies what it is (Figure 3, Table 5). For eager PyTorch, one scoring call costs 20.0 ms of fixed overhead plus 0.10 ms per hundred candidates: 99% of a full-size call is fixed. The call is a sequence of a few hundred small kernels, each launched from Python, and each finishes before the next is issued, so its time is set by the number of launches, not by the number of candidates. Compiling the call with `torch.compile` and replaying it as a CUDA Graph cuts the fixed part to 1.9 ms, but the compiled planner still returns to the CPU every iteration, and its single-robot search still takes 82.3 ms for arithmetic worth a few milliseconds.

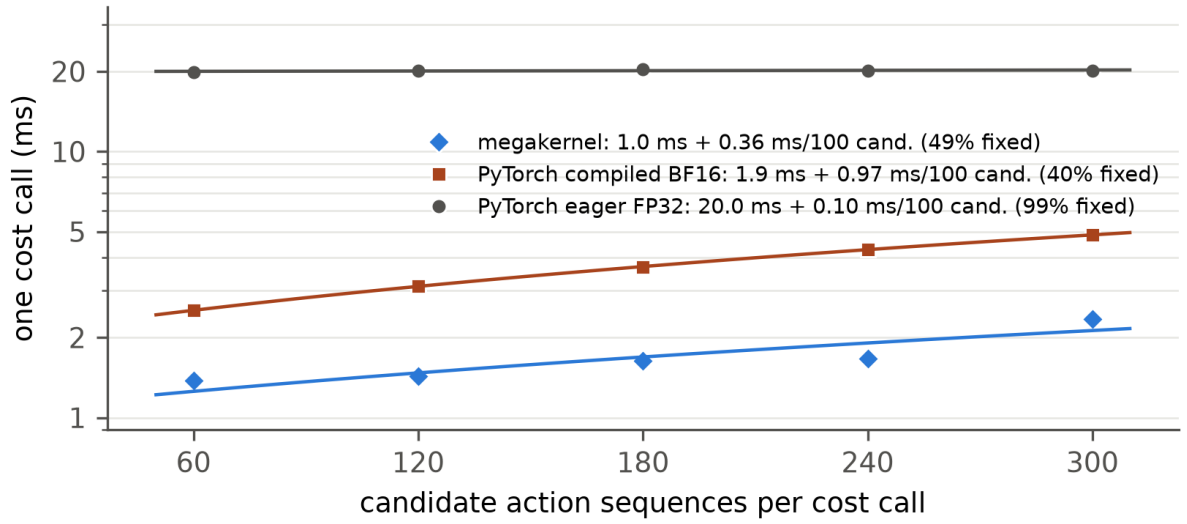


Figure 3. Latency of one scoring call (a five-step rollout of every candidate) against the number of candidates, with the fitted line latency = a + b·N for each execution method. Eager PyTorch is almost entirely fixed overhead; the megakernel's call is shown for comparison (Section 6.5). Medians of 30 to 50 calls on one H100.

Table 5. Fitted scaling model of one scoring call, and what it predicts iCEM saves (30 iterations in both schedules: CEM 9,000 rollouts, iCEM 2,820).

execution	fixed cost a (ms)	per 100 candidates b (ms)	R^2	fixed share at 300	predicted iCEM speed-up
PyTorch eager FP32 (reference)	19.96	0.095	0.26	0.99	1.01×
PyTorch compiled BF16 (optimised baseline)	1.95	0.971	1.00	0.40	1.70×
megakernel, one call per launch	1.04	0.362	0.79	0.49	1.54×

The low R^2 for eager execution is itself the finding: there is almost no variation with the number of candidates for a line to explain.

4.4 Finding 3: sample-efficient search only pays once overhead is gone

iCEM evaluates 3.19 times fewer candidates than CEM, and the scaling model predicts what that is worth. Under eager execution, where 99% of each call is fixed, it predicts a speed-up of 1.01, that is, nothing. The probes confirm it: encoding once per plan, the eager planner takes 377 ms with CEM and 410 ms with iCEM, 0.92 times as fast, because iCEM's bookkeeping costs slightly more than its smaller batches save. Only an execution method with a small fixed cost can turn fewer candidates into less time. A sample-efficient algorithm and an efficient execution model are therefore complements, not alternatives, and the design must provide both.

4.5 From findings to design

Table 6 maps each finding to the design response of Section 5.

Table 6. Design responses to the profile.

finding	evidence	design response
58 of 60 encoder calls per plan are redundant	telemetry; 207 ms, 33%	encode each image once per plan; cache the goal for the whole episode

finding	evidence	design response
the search is fixed overhead: launches, Python, CPU round trips	99% fixed share; 409 µs per layer for about 3 µs of arithmetic; 45 ms of synchronisation	run all 30 iterations, including sampling, scoring and distribution updates, inside one GPU program
fewer candidates only help without the overhead	predicted 1.01× and observed 0.92× for iCEM in eager PyTorch	adopt iCEM together with, not instead of, the fused execution
the work is a long dependent chain of tiny steps	900 layer evaluations per plan, 64 to 300 rows each	minimise the cost of each step: keep data on chip, cheap synchronisation, weights prefetched ahead of use
encoding becomes a visible share once the search is fast	6.9 ms of PyTorch encoding against a 41 ms search	move the encoder into the same GPU program

5. Solution design and development

The solution, LeWM-MK, combines four changes, each a response to a finding of Section 4. The trained model is used unchanged, and the planner keeps the reference's horizon, iteration count and elite count.

5.1 Encode each image once

The observation and the goal are encoded once per plan and passed to the search as embeddings, and the search keeps its ranking and distribution updates on the GPU, so that the CPU waits only for the final result. Within an episode the goal image never changes, so its embedding is computed on the first plan and reused, keyed by a hash of the image's bytes. This change is exact: on fixed candidates it reproduces the reference's scores with zero difference.

5.2 Search with iCEM

The search uses iCEM with the schedule that an earlier study of this planner validated on Push-T (Appendix C): 300 candidates in the first iteration, shrinking by a factor of 1.25 per iteration to a floor of 60 (twice the elite count), rounded up to multiples of 60 so that only five batch shapes occur; the top 30 candidates as elites; and the best 5 elites of each iteration carried into the next. A plan evaluates 2,820 rollouts instead of 9,000. Coloured noise, momentum and action clipping, which iCEM also offers, reduced success on Push-T and are not used. As Finding 3 showed, this change saves time only in combination with the next.

5.3 One GPU program per plan

The core of the solution is a persistent kernel that runs a whole plan: all 30 iterations of sampling candidates, encoding their actions, computing the AdaLN modulation, rolling every candidate through the predictor for five steps, scoring, selecting the elites and refitting the distribution. The CPU launches it once and receives the final action distribution. Figure 4 shows its structure. It is written with ThunderKittens for the H100 (Spector, Arora, et al., 2025).

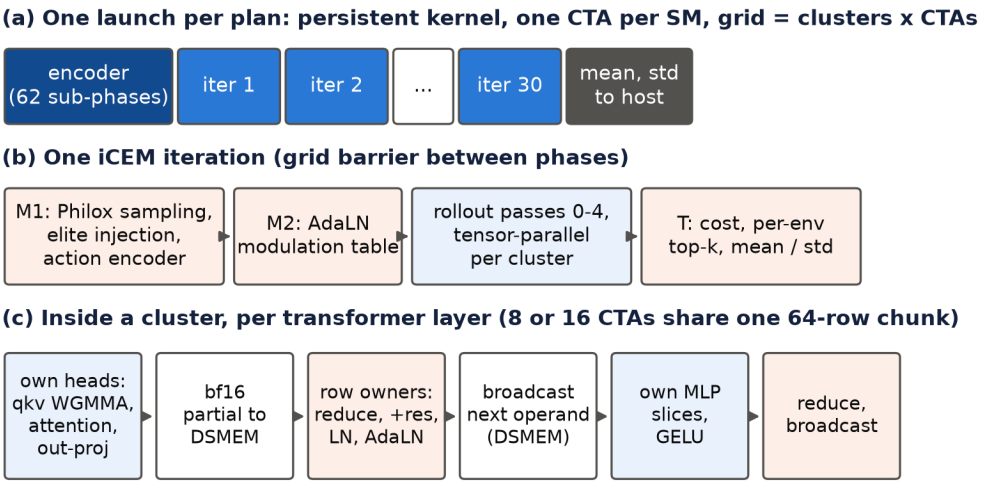


Figure 4. Structure of LeWM-MK. (a) One launch per plan, beginning with the encoder (Section 5.4). (b) The phases of one iCEM iteration, separated by barriers across the whole GPU. (c) One transformer layer inside a cluster of 8 or 16 processors: each computes its share of the attention heads and MLP, owner processors combine the partial results through distributed shared memory, apply the residual connection, normalisation and AdaLN modulation, and broadcast the input of the next operation.

Four design decisions carry its speed, and each attacks a cost identified by the profile (Table 7).

The whole GPU stays resident. The kernel starts one block of 256 threads on each of 112 of the H100's 132 processors (streaming multiprocessors), as seven clusters of 16, and keeps them running for the whole plan, so no kernel is ever launched mid-plan. Candidates are drawn inside the kernel from a counter-based random number generator (Salmon et al., 2011), and the elites are selected inside the kernel, so the CPU is not consulted between iterations. Phases are separated by a barrier across the whole GPU, which costs 1.6 µs.

One small model is spread across a cluster. At one robot the matrices are only 64 to 300 rows tall, too small to occupy a whole GPU one operation at a time. The kernel therefore groups processors into clusters of 16 and splits every layer across a cluster: each processor owns one of the 16 attention heads and a sixteenth of the MLP, computes its share with the H100's tensor-core instructions, and exchanges partial results with its neighbours through distributed shared memory rather than main memory. The cluster barrier that orders those exchanges is on the critical path four times per layer, 3,600 times per plan. Implemented naively, with full memory ordering on every signal, it cost 3.6 µs; with one memory fence and relaxed signals it costs 0.65 µs. Together with a faster GELU, this took about 11 ms off the search.

Weights stream ahead of use. Because the planner's sequence of operations is fixed, the order in which weights are needed is known before the plan starts. The weights are folded and repacked once on the CPU into 1,472 blocks of 24 KB, already laid out exactly as the tensor-core instructions read them, and each group of threads keeps a two-slot buffer that it refills with an asynchronous bulk copy as soon as it releases a slot. Prefetching therefore continues across operation, layer and iteration boundaries, and the computation almost never waits for weights. No separate thread group is spent on copying: on the H100 a ninth

group of threads would cap every thread's registers at 168 instead of 255 and force spills to slower memory.

No redundant work in the rollout. In the first three of the five rollout steps the predictor's history window is still growing, so the attention keys and values of earlier steps are exactly those already computed; the kernel caches them and computes 9 token rows per candidate instead of 12, with no approximation.

Table 7. Key design decisions of the kernel and their measured effect.

decision	cost it removes	measured effect
persistent kernel, in-kernel sampling and elite selection	kernel launches and CPU round trips between and within iterations	one launch per plan; grid barrier 1.6 µs, empty launch 14 µs
tensor parallelism across a 16-processor cluster	idle processors at small matrix sizes; traffic through main memory	partial results exchanged on chip through distributed shared memory
relaxed-ordering cluster barrier	serialised signals on the critical path (3,600 per plan)	3.6 → 0.65 µs per barrier; search 55.3 → 43.8 ms together with a faster GELU
static weight stream without a copy thread group	waiting for weights; register spills	255 registers per thread instead of 168; search 43.8 → 41.9 ms
exact reuse of cached attention keys and values	recomputing earlier rollout steps	9 instead of 12 token rows per candidate

The kernel reached its final speed through a development loop that re-profiled each change. Three further attempts (removing the first accumulation of each matrix chain, prefetching cached keys and values, and prefetching owner operands across barriers) all made it slower by raising register pressure and were discarded.

5.4 The encoder inside the same program

Once the search took about 41 ms, the 6.9 ms PyTorch encoder and the CPU image transform before it became a visible share of a plan. The encoder therefore runs as the first phase of the same kernel. The kernel reads the raw 8-bit camera frame, normalises it and computes the patch embedding as one matrix multiplication. It then runs the 12 ViT layers with an attention computation that never stores the full attention matrix (Dao et al., 2022), and applies the projector. Encoding the observation now takes 1.25 ms (1.34 ms with the goal on an episode's first plan). A plan is one GPU launch from camera frame to action distribution.

5.5 Numerical design

The kernel computes matrix products with 16-bit brain-float (BF16) inputs and 32-bit (FP32) accumulation, keeps the residual stream, normalisation statistics, scores and search statistics in FP32, and uses an accurate approximation of the error function in the GELU activation (Abramowitz & Stegun, 1964). This is the same numerical class as the BF16 arithmetic of the optimised PyTorch baseline, which mixed-precision practice treats as safe for inference (Micikevicius et al., 2018; Kalamkar et al., 2019). The kernel is mathematically equivalent to the reference but not bit-identical, so its acceptance rests on measured error and on closed-loop behaviour (Section 6).

5.6 Deployment

LeWM-MK is deployed as a planner class (`MegakernelICEMSolver`) that plugs into the stable-worldmodel evaluation harness through its configuration system, so a user switches planners by changing one configuration name. The kernel is compiled once for the H100 (`sm_90a`), and builds are content-addressed by the hash of the source so that a stale build can never run. The analysis is deployed as the solution program described in Appendix B.

6. Evaluation

6.1 Latency

Table 8 and Figure 5 compare the three systems on the same H100. LeWM-MK computes a plan for one robot in 46.5 ms, 13.4 times faster than the reference planner and 3.9 times faster than the optimised PyTorch planner, which already uses compilation, CUDA Graph replay and BF16 arithmetic. All three confidence intervals are narrow and far apart.

Table 8. End-to-end planning latency for one robot on one H100 (ms). Speed-ups are ratios of medians with bootstrap 95% intervals.

system	plans	median [95% CI]	90th percentile	interquartile range	step call	speed-up of LeWM-MK
reference planner	73	622.4 [605.5, 626.7]	673.8	595.9 – 641.7	2.14	13.4× [13.0, 13.5]
optimised PyTorch planner	72	182.1 [177.3, 185.7]	204.3	175.3 – 195.4	1.51	3.9× [3.8, 4.0]
LeWM-MK	76	**46.5 [46.4, 46.6]**	**47.0**	**46.1 – 46.8**	**0.82**	—

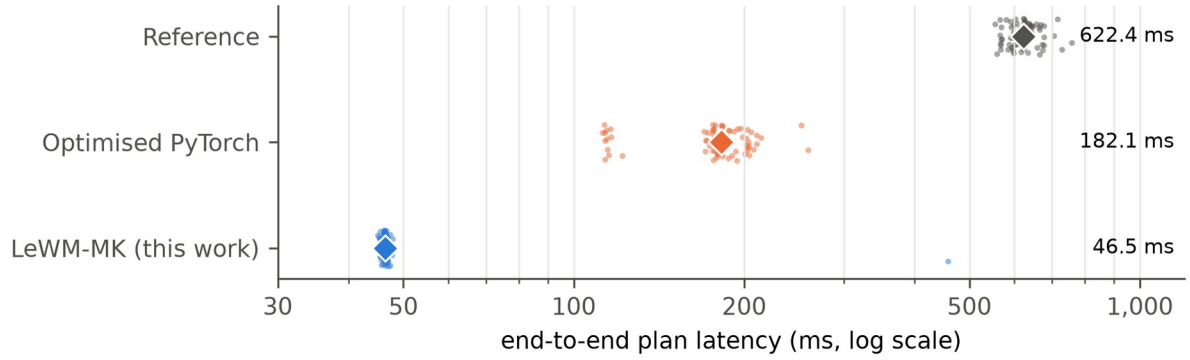


Figure 5. End-to-end planning latency of the three systems, log scale. Small points are individual plans; diamonds are medians.

For a controller, predictability matters as much as speed. The reference planner's plans spread over 200 ms and the optimised PyTorch planner's over 150 ms, with two distinct clusters. LeWM-MK's interquartile range is 0.7 ms wide, and its 90th percentile is within 0.6 ms of its median. One of its 76 plans took 458 ms, a one-off in a single process that the data do not explain; it is reported and kept. At the median, a LeWM-MK plan fits inside the 50 ms period of a 20 Hz control loop, and comfortably inside the 100 ms of a 10 Hz loop. The calls that only execute a planned action also became cheaper (0.82 ms), because the fused pipeline no longer runs the CPU image transform on every call.

6.2 The new compute profile

Figure 6 shows the profile before and after. Of LeWM-MK's 46.5 ms, the search takes 41.3 ms (89%), the encoder 1.3 ms (3%) and everything outside the kernel 3.9 ms (8%). The redundant encoding is gone, the CPU synchronisation is gone, and the remaining search costs 46 µs per layer evaluation against 409 µs in the reference, nine times less.

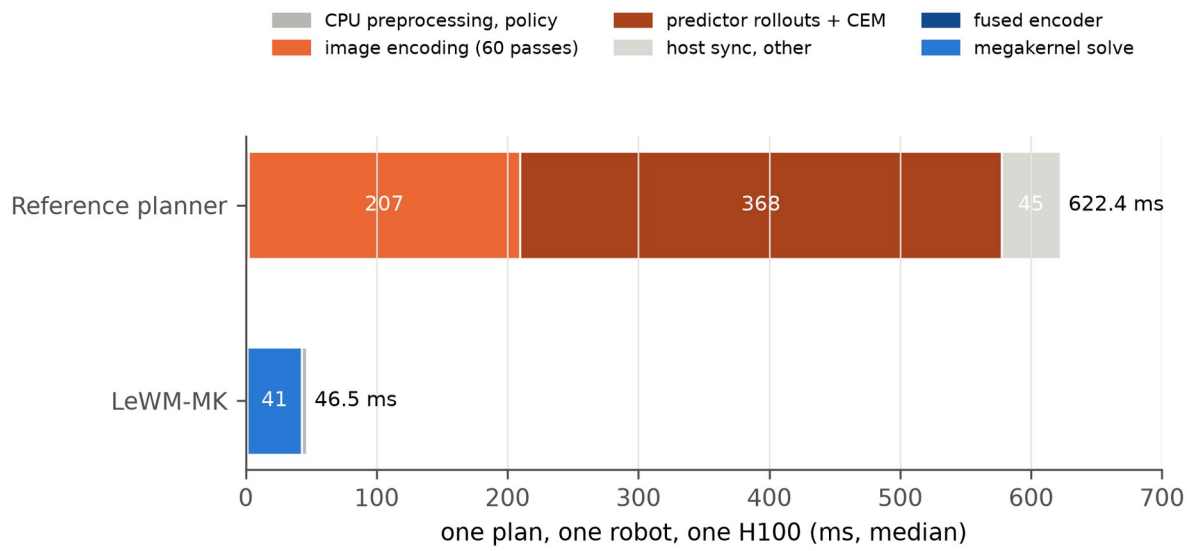


Figure 6. Compute profile of one plan before (reference planner) and after (LeWM-MK), by stage. Medians; stages without a direct measurement are attributed by difference (Section 3.5).

What limits the search now is the depth of the chain itself. The measured synchronisation primitives account for about 2.8 ms of the 41.3 ms (four cluster barriers per layer at 0.68 µs, and the grid barriers between phases at 1.6 µs), or 7%. The rest is the latency of the tensor-core instruction chains and the work between them at small matrix sizes. The weight stream almost never waits. LeWM-MK has therefore moved the planner from an overhead-bound regime into one bounded by the latency of its own dependent arithmetic, which is the regime a single robot's planning problem imposes.

6.3 Task success

Faster planning must not change what the robot achieves. Figure 7 and Table 9 give the evidence on the 300 paired Push-T episodes. The three systems solve 261, 261 and 260 episodes, and their Wilson intervals overlap almost completely. LeWM-MK and the reference planner disagree on 39 of the 300 episodes: LeWM-MK solves 19 that the reference fails, and fails 20 that the reference solves. The paired difference is −0.3 percentage points with a 95% interval of −4.3 to +3.7, and the exact McNemar test gives p = 1.00. The reference's success rate of 87% is below the 96% that LeWM's authors report for Push-T; we used the evaluation configuration released with the code and did not investigate the difference, which applies equally to every system.

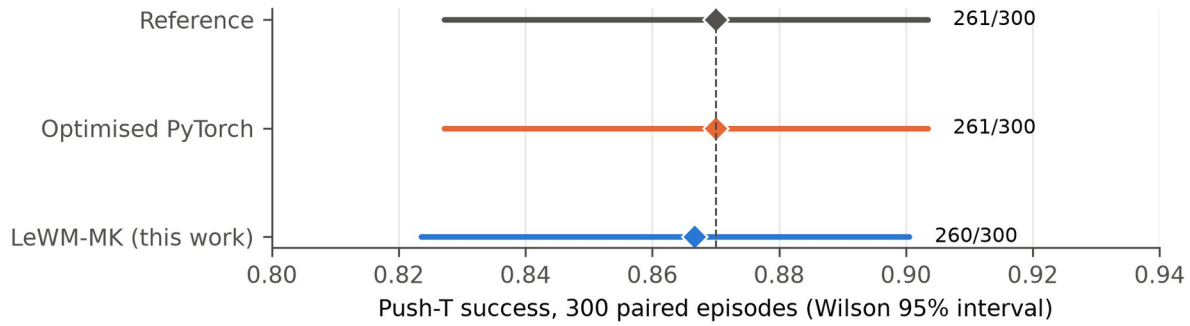


Figure 7. Push-T success of the three systems on the same 300 episodes, with Wilson 95% intervals. The dashed line is the reference planner.

Table 9. Paired comparison of task success on the 300 episodes. Wins and losses count the episodes solved by only one of the two systems; the interval is a paired bootstrap (10,000 resamples); p is the exact McNemar test.

comparison	successes	difference (points) [95% CI]	wins / losses	p
LeWM-MK vs reference	260 vs 261	−0.3 [−4.3, +3.7]	19 / 20	1.00
optimised PyTorch vs reference	261 vs 261	0.0 [−4.0, +4.0]	18 / 18	1.00
LeWM-MK vs optimised PyTorch	260 vs 261	−0.3 [−4.0, +3.3]	16 / 17	1.00

The 39 disagreements are expected. Planning is a randomised search, and an episode that sits near the boundary between success and failure is decided by which candidates happen to be drawn; LeWM-MK draws its candidates from its own random number generator, so it redraws those episodes. The optimised PyTorch planner, which also differs from the reference only in its search and arithmetic, disagrees with it on 36. The evaluation detects differences of about 6 percentage points at 80% power. Within that resolution, LeWM-MK does not change task success.

6.4 Numerical fidelity

Table 10 compares LeWM-MK's arithmetic with FP32 on identical inputs. Across eight scoring cases its largest relative error is 0.06% to 0.32%, against 0.05% to 0.40% for the optimised PyTorch planner's BF16 arithmetic, and the two agree with FP32 on 27 to 30 and 28 to 30 of the 30 best candidates respectively. The fused encoder is closer to FP32 than PyTorch's own BF16 encoder: its embeddings differ by 0.41% (relative L2 norm) against 0.52%, and plans scored from its embeddings stay within 0.32% of FP32 with identical top-30 sets.

Table 10. Numerical agreement with FP32 on identical inputs (one H100).

check	optimised PyTorch (BF16)	LeWM-MK
scoring, maximum relative error (8 cases)	0.05% – 0.40%	0.06% – 0.32%
scoring, agreement of the 30 best candidates	28 – 30 of 30	27 – 30 of 30
encoder embedding, relative L2 error (32 frames)	0.52%	0.41%
scoring from encoder embeddings, maximum relative error / top-30 (6 cases)	—	0.32% / 30 of 30

6.5 Validating the scaling model

The scaling model of Section 4.3 made a prediction that the probes can test. Table 11 compares the predicted and observed value of iCEM in the two execution regimes.

Table 11. Predicted and observed speed-up of iCEM over CEM (end-to-end medians of the probes, encoder in PyTorch in both kernel probes).

execution	fixed share of a scoring call	predicted	observed
PyTorch eager (377.0 vs 410.2 ms)	99%	1.01×	0.92×
LeWM-MK search (82.1 vs 59.3 ms)	49%	1.54×	1.39×

The model gets the qualitative answer right in both regimes: no gain where the cost is fixed, a substantial gain where it is not. It overestimates both, by about 0.1, because it ignores costs that do not depend on the number of candidates: iCEM's elite bookkeeping, and inside the kernel the 30-iteration chain and the 64-row tiles that are partly empty at 60 candidates. As an analytic tool it did what was needed. It explained why the sample-efficient algorithm had appeared useless, and it predicted that it would pay once the overhead was removed.

7. Discussion

7.1 Were the objectives met?

O1 (profile). Yes. The profile located the reference planner's time precisely: a third in redundant image encoding, most of the rest in a search whose cost is 99% fixed overhead per call, and 7% in synchronisation with the CPU. The arithmetic the planner exists to perform accounts for a small fraction of its time.

O2 (design and implement). Yes. LeWM-MK removes each bottleneck the profile identified, with the trained model unchanged and the planner's horizon, iteration count and elite count unchanged. The one algorithmic change, iCEM's shrinking population, was adopted because the profile showed when it would pay.

O3 (evaluate). Yes. One plan for one robot takes 46.5 ms against 622.4 ms for the reference (13.4×) and 182.1 ms for the optimised PyTorch planner (3.9×), with a 90th percentile of 47.0 ms. On 300 paired episodes task success does not change detectably (−0.3 points, 95% CI −4.3 to +3.7), and the kernel's numerical error matches that of the accepted BF16 arithmetic.

O4 (deploy). Yes. The planner plugs into the existing harness by configuration name, and the solution program regenerates every result in this report from the collected data (Appendix B).

7.2 Why the solution works

The planner of a small world model is an unusual workload. Its model is tiny and its computation is deep: a few hundred billion floating-point operations per plan, spread across 900 dependent steps of 64 to 300 rows each. Neither of the quantities by which AI inference is usually optimised, the amount of arithmetic and the amount of memory traffic, limits it. What limits it is the fixed cost paid at every step: a kernel launch, a line of Python, a trip to the CPU, a round trip through main memory. Lower precision and bigger batches, the usual remedies, do little at one robot, because they shrink costs that were not binding. The remedy that works is to remove the steps between the arithmetic, and the planner's structure allows it. Once the search schedule is fixed, the planner's sequence of operations, the weights it needs and the synchronisation it requires are all known before it starts, which is what makes a single precomputed GPU program possible.

The analysis also showed that an algorithmic improvement and an execution improvement can depend on each other. iCEM was worth nothing under eager execution and a 1.39-times speed-up under fused execution. An evaluation that measured the algorithm in only one execution model would have reached the wrong conclusion either way. A planner's speed is a property of the algorithm and its execution together, and the profiling data are what reveal the interaction.

7.3 Position relative to other approaches

Other routes to faster world-model planning change the model or remove the search. Fast LeWorldModel retrains the predictor to roll out a whole candidate in one pass and halves planning time (Gao & Xu, 2026). An inverse-dynamics policy trained over LeWM replaces search with a single network evaluation, at 100 to 130 times lower cost per decision (Nguyen et al., 2026). LeWM-MK keeps the released weights and the explicit search and changes

only how they are executed, so it composes with both: a faster model would shorten the chain the kernel walks, and a learned policy could give the search a better starting point. Megakernels for language models fuse one forward pass of a model per launch (Spector, Juravsky, Sul, Dugan, et al., 2025; Cheng et al., 2026). LeWM-MK fuses an entire optimisation loop of 2,820 model evaluations, 30 rounds of elite selection and their updates, which to our knowledge has not been done before for a learned world model.

7.4 Limitations

Statistical resolution. Three hundred paired episodes detect differences in task success of about 6 percentage points. The claim that LeWM-MK preserves success is a claim of no detectable change at that resolution; a difference of 2 or 3 points either way would need about 2,500 paired episodes to resolve.

Attribution by difference. Two stages of the reference profile (the search and the synchronisation) and one of the new profile (the time outside the kernel) are estimated from combinations of measurements rather than measured directly. The micro-benchmark also measures an eager scoring call at 20 ms where the harness implies about 12 ms per iteration. The analysis relies on the shape of the scaling curve, which both sources agree on, not on its absolute level.

Scope. All results are for one model, one task and one GPU type. The kernel is written for LeWM's dimensions and for features specific to the H100, and does not run on other GPUs without porting. The mechanism, a latency-bound chain of small dependent steps, is shared by sampling-based planning with any small world model, so the analysis should transfer even where the numbers do not.

Simulation and the planning call only. Push-T is simulated, and a 46.5 ms plan is a statement about computation, not about control on a physical robot, which adds sensing and actuation delays. Only the planning call was optimised.

Measurement variance. Latency was measured in one pass of 61 episodes per system, in a rented cloud container. An earlier pass of the same benchmark gave PyTorch medians that differed by up to 18% (Appendix C), while the configurations running the kernel reproduced to within about 2%. The PyTorch medians are therefore best read as typical values, and so are the speed-ups that use them.

8. Conclusion and future work

8.1 Conclusion

This project set out to make planning with a learned world model fast enough for robot control, without retraining the model or making the robot less successful, and to do it by analysing data rather than by guesswork. Profiling LeWM's released planner on an H100 showed that its 622 ms per plan was mostly not computation. A third was spent re-encoding two images 60 times. Most of the rest was a search whose cost per iteration is 99% fixed overhead, which also explains why a sample-efficient search algorithm had appeared useless. The arithmetic the planner exists to perform took a small fraction of its time.

The solution follows from that profile. Each image is encoded once. The search uses iCEM, which evaluates 3.2 times fewer candidates. The whole plan, from camera frame to action distribution, runs as one persistent GPU program that keeps its data on chip, spreads the small model across a cluster of processors and streams its weights ahead of use. One plan for one robot now takes 46.5 ms with a 90th percentile of 47.0 ms: 13.4 times faster than the reference and 3.9 times faster than an optimised PyTorch implementation on the same GPU. Task success on 300 paired episodes is unchanged within the evaluation's resolution, and the numerical error matches the accepted half-precision arithmetic. Planning with this world model has moved from the time scale of deliberation to the time scale of control.

The broader lesson is analytic. When an AI system is slow, the remedy depends on what kind of slow it is. A small model planning for one robot is limited neither by arithmetic nor by memory bandwidth but by fixed costs repeated thousands of times. A simple fitted model of how latency scales with workload is enough to tell the cases apart, and it predicted correctly which optimisation would pay.

8.2 Future work

Spend the saved time on better plans. The search budget was held fixed to measure execution alone. At 46.5 ms a plan with more iterations or a larger population still fits a 100 ms control period. A sweep of these settings, now cheap to run, would map the trade-off between latency and success and test whether more search closes the gap between the 87% success measured here and the 96% that LeWM's authors report.

Shorten the remaining chain. The search is now limited by 900 dependent layer evaluations with several cross-processor exchanges each. Combining the kernel with Fast LeWorldModel's one-pass rollout (Gao & Xu, 2026), or redesigning the kernel so that each cluster holds one layer and candidates stream through, would attack the chain directly.

Other models, other hardware. LeWM's single-vector representation is the most favourable case for this design. Larger world models such as DINO-WM and V-JEPA 2-AC have larger matrices and may return to a regime where throughput matters, and onboard robot computers have different costs again. Automatic megakernel compilers (Cheng et al., 2026; Jin et al., 2026) could make porting a matter of compilation rather than rewriting.

Evaluate at the resolution a deployment needs, and on a robot. Resolving differences in success of 2 points needs about 2,500 paired episodes, which at the new speed is a matter of minutes of planning rather than hours. The final step is a physical robot, where the question becomes how much faster replanning improves what the robot achieves.

References

Abramowitz, M., & Stegun, I. A. (1964). *Handbook of mathematical functions with formulas, graphs, and mathematical tables* (National Bureau of Standards Applied Mathematics Series 55). U.S. Government Printing Office.

Andersch, M., Palmer, G., Krashinsky, R., Stam, N., Mehta, V., Brito, G., & Ramaswamy, S. (2022, March 22). *NVIDIA Hopper architecture in-depth*. NVIDIA Technical Blog. https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/

Ansel, J., Yang, E., He, H., Gimelshein, N., Jain, A., Voznesensky, M., Bao, B., Bell, P., Berard, D., Burovski, E., Chauhan, G., Chourdia, A., Constable, W., Desmaison, A., DeVito, Z., Ellison, E., Feng, W., Gong, J., Gschwind, M., . . . Chintala, S. (2024). PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (pp. 929–947). ACM. https://doi.org/10.1145/3620665.3640366

Assran, M., Bardes, A., Fan, D., Garrido, Q., Howes, R., Komeili, M., Muckley, M., Rizvi, A., Roberts, C., Sinha, K., Zholus, A., Arnaud, S., Gejji, A., Martin, A., Hogan, F. R., Dugas, D., Bojanowski, P., Khalidov, V., Labatut, P., . . . Ballas, N. (2025). *V-JEPA 2: Self-supervised video models enable understanding, prediction and planning* (arXiv:2506.09985). arXiv. https://doi.org/10.48550/arXiv.2506.09985

Assran, M., Duval, Q., Misra, I., Bojanowski, P., Vincent, P., Rabbat, M., LeCun, Y., & Ballas, N. (2023). Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 15619–15629). IEEE. https://doi.org/10.1109/CVPR52729.2023.01499

Balestriero, R., & LeCun, Y. (2025). *LeJEPA: Provable and scalable self-supervised learning without the heuristics* (arXiv:2511.08544). arXiv. https://doi.org/10.48550/arXiv.2511.08544

Bardes, A., Garrido, Q., Ponce, J., Chen, X., Rabbat, M., LeCun, Y., Assran, M., & Ballas, N. (2024). Revisiting feature prediction for learning visual representations from video. *Transactions on Machine Learning Research*. https://openreview.net/forum?id=QaCCuDfBk2

Bhardwaj, M., Sundaralingam, B., Mousavian, A., Ratliff, N. D., Fox, D., Ramos, F., & Boots, B. (2022). STORM: An integrated framework for fast joint-space model-predictive control for reactive manipulation. In *Proceedings of the 5th Conference on Robot Learning* (Vol. 164, pp. 750–759). PMLR. https://proceedings.mlr.press/v164/bhardwaj22a.html

Cheng, X., Zhang, Z., Zhou, Y., Ji, J., Jiang, J., Zhao, Z., Xiao, Z., Ye, Z., Huang, Y., Lai, R., Jin, H., Hou, B., Wu, M., Dong, Y., Yip, A., Wang, S., Yang, W., Miao, X., Chen, T., & Jia, Z. (2026). MPK: A compiler and runtime for mega-kernelizing tensor programs. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)* (pp. 1909–1926). USENIX Association. https://www.usenix.org/conference/osdi26/presentation/cheng

Chua, K., Calandra, R., McAllister, R., & Levine, S. (2018). Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *Advances in Neural Information Processing Systems 31* (pp. 4754–4765). Curran Associates. https://proceedings.neurips.cc/paper/2018/hash/3de568f8597b94bda53149c7d7f5958c-Abstract.html

Connor, R. J. (1987). Sample size for testing differences in proportions for the paired-sample design. *Biometrics, 43*(1), 207–211. https://doi.org/10.2307/2531961

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems 35* (pp. 16344–16359). Curran Associates. https://doi.org/10.52202/068431-1189

de Boer, P.-T., Kroese, D. P., Mannor, S., & Rubinstein, R. Y. (2005). A tutorial on the cross-entropy method. *Annals of Operations Research, 134*(1), 19–67. https://doi.org/10.1007/s10479-005-5724-z

Diamos, G., Sengupta, S., Catanzaro, B., Chrzanowski, M., Coates, A., Elsen, E., Engel, J., Hannun, A., & Satheesh, S. (2016). Persistent RNNs: Stashing recurrent weights on-chip. In *Proceedings of the 33rd International Conference on Machine Learning* (Vol. 48, pp. 2024–2033). PMLR. https://proceedings.mlr.press/v48/diamos16.html

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*. https://openreview.net/forum?id=YicbFdNTTy

Efron, B., & Tibshirani, R. J. (1993). *An introduction to the bootstrap*. Chapman & Hall. https://doi.org/10.1201/9780429246593

Gao, Y., & Xu, X. (2026). *Fast LeWorldModel* (arXiv:2606.26217). arXiv. https://doi.org/10.48550/arXiv.2606.26217

Gray, A. (2019, September 5). *Getting started with CUDA Graphs*. NVIDIA Technical Blog. https://developer.nvidia.com/blog/cuda-graphs/

Gupta, K., Stuart, J. A., & Owens, J. D. (2012). A study of persistent threads style GPU programming for GPGPU workloads. In *2012 Innovative Parallel Computing (InPar)* (pp. 1–14). IEEE. https://doi.org/10.1109/InPar.2012.6339596

Hafner, D., Lillicrap, T., Fischer, I., Villegas, R., Ha, D., Lee, H., & Davidson, J. (2019). Learning latent dynamics for planning from pixels. In *Proceedings of the 36th International Conference on Machine Learning* (Vol. 97, pp. 2555–2565). PMLR. https://proceedings.mlr.press/v97/hafner19a.html

Hansen, N. A., Su, H., & Wang, X. (2022). Temporal difference learning for model predictive control. In *Proceedings of the 39th International Conference on Machine Learning* (Vol. 162, pp. 8387–8406). PMLR. https://proceedings.mlr.press/v162/hansen22a.html

Huang, K., Lale, S., Rosolia, U., Shi, Y., & Anandkumar, A. (2021). *CEM-GD: Cross-entropy method with gradient descent planner for model-based reinforcement learning* (arXiv:2112.07746). arXiv. https://doi.org/10.48550/arXiv.2112.07746

Ivanov, A., Dryden, N., Ben-Nun, T., Li, S., & Hoefler, T. (2021). Data movement is all you need: A case study on optimizing transformers. In *Proceedings of Machine Learning and Systems 3* (MLSys 2021). https://doi.org/10.48550/arXiv.2007.00072

Jin, H., Hou, B., Wang, G., Lai, R., Chen, J., Ye, Z., Cai, Y., Dong, Y., Cheng, X., Zhang, Z., Zhao, Y., Huang, Y., Yang, L., Jiang, J., Oliaro, G., Ji, J., Miao, X., Grover, V., Mowry, T. C., . . . Chen, T. (2026). *Event Tensor: A unified abstraction for compiling dynamic megakernel* (arXiv:2604.13327). arXiv. https://doi.org/10.48550/arXiv.2604.13327

Kalamkar, D., Mudigere, D., Mellempudi, N., Das, D., Banerjee, K., Avancha, S., Vooturi, D. T., Jammalamadaka, N., Huang, J., Yuen, H., Yang, J., Park, J., Heinecke, A., Georganas, E., Srinivasan, S., Kundu, A., Smelyanskiy, M., Kaul, B., & Dubey, P. (2019). *A study of BFLOAT16 for deep learning training* (arXiv:1905.12322). arXiv. https://doi.org/10.48550/arXiv.1905.12322

Luo, W., Fan, R., Li, Z., Du, D., Wang, Q., & Chu, X. (2024). Benchmarking and dissecting the Nvidia Hopper GPU architecture. In *2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS)* (pp. 656–667). IEEE. https://doi.org/10.1109/IPDPS57955.2024.00064

Maes, L., Le Lidec, Q., Facury, L., Massaudi, N., Chaurasia, A., Capuano, F., Gao, R., Gillin, T., Haramati, D., Scieur, D., LeCun, Y., & Balestriero, R. (2026). *stable-worldmodel: A platform for reproducible world modeling research and evaluation* (arXiv:2605.21800). arXiv. https://doi.org/10.48550/arXiv.2605.21800

Maes, L., Le Lidec, Q., Scieur, D., LeCun, Y., & Balestriero, R. (2026). *LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels* (arXiv:2603.19312). arXiv. https://doi.org/10.48550/arXiv.2603.19312

McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika, 12*(2), 153–157. https://doi.org/10.1007/BF02295996

Micikevicius, P., Narang, S., Alben, J., Diamos, G., Elsen, E., Garcia, D., Ginsburg, B., Houston, M., Kuchaiev, O., Venkatesh, G., & Wu, H. (2018). Mixed precision training. In *International Conference on Learning Representations*. https://openreview.net/forum?id=r1gs9JgRZ

Nguyen, H., Xu, X., & Huang, X. (2026). *Latent geometry beyond search: Amortizing planning in world models* (arXiv:2605.08732). arXiv. https://doi.org/10.48550/arXiv.2605.08732

Nrusimha, A., Brandon, W., Mishra, M., Shen, Y., Panda, R., Ragan-Kelley, J., & Kim, Y. (2025). *FlashFormer: Whole-model kernels for efficient low-batch inference* (arXiv:2505.22758). arXiv. https://doi.org/10.48550/arXiv.2505.22758

Oquab, M., Darcet, T., Moutakanni, T., Vo, H. V., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El-Nouby, A., Assran, M., Ballas, N., Galuba, W., Howes, R., Huang, P.-Y., Li, S.-W., Misra, I., Rabbat, M., Sharma, V., . . . Bojanowski, P. (2024). DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*. https://openreview.net/forum?id=a68SUt6zFt

Peebles, W., & Xie, S. (2023). Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)* (pp. 4172–4182). IEEE. https://doi.org/10.1109/ICCV51070.2023.00387

Pinneri, C., Sawant, S., Blaes, S., Achterhold, J., Stueckler, J., Rolinek, M., & Martius, G. (2021). Sample-efficient cross-entropy method for real-time planning. In *Proceedings of the 2020 Conference on Robot Learning* (Vol. 155, pp. 1049–1065). PMLR. https://proceedings.mlr.press/v155/pinneri21a.html

Pope, R., Douglas, S., Chowdhery, A., Devlin, J., Bradbury, J., Heek, J., Xiao, K., Agrawal, S., & Dean, J. (2023). Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems 5* (MLSys 2023). https://proceedings.mlsys.org/paper_files/paper/2023/hash/c4be71ab8d24cdfb45e3d06dbfca2780-Abstract-mlsys2023.html

Rubinstein, R. (1999). The cross-entropy method for combinatorial and continuous optimization. *Methodology and Computing in Applied Probability, 1*(2), 127–190. https://doi.org/10.1023/A:1010091220143

Salmon, J. K., Moraes, M. A., Dror, R. O., & Shaw, D. E. (2011). Parallel random numbers: As easy as 1, 2, 3. In *Proceedings of the 2011 International Conference for High Performance Computing, Networking, Storage and Analysis* (Article 16). ACM. https://doi.org/10.1145/2063384.2063405

Shah, J., Bikshandi, G., Zhang, Y., Thakkar, V., Ramani, P., & Dao, T. (2024). FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. In *Advances in Neural Information Processing Systems 37* (pp. 68658–68685). Curran Associates. https://doi.org/10.52202/079017-2193

Sobal, U., Zhang, W., Cho, K., Balestriero, R., Rudner, T. G. J., & LeCun, Y. (2025). Learning from reward-free offline data: A case for planning with latent dynamics models. In *Advances in Neural Information Processing Systems 38* (pp. 49126–49162). Curran Associates. https://doi.org/10.52202/085713-1465

Spector, B., Juravsky, J., Sul, S., Dugan, O., Lim, D., Fu, D., Arora, S., & Ré, C. (2025, May 27). *Look ma, no bubbles! Designing a low-latency megakernel for Llama-1B*. Hazy Research Blog, Stanford University. https://hazyresearch.stanford.edu/blog/2025-05-27-no-bubbles

Spector, B. F., Arora, S., Singhal, A., Parthasarathy, A., Fu, D. Y., & Ré, C. (2025). ThunderKittens: Simple, fast, and *adorable* kernels. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=0fJfVOSUra

Sundaralingam, B., Hari, S. K. S., Fishman, A., Garrett, C., Van Wyk, K., Blukis, V., Millane, A., Oleynikova, H., Handa, A., Ramos, F., Ratliff, N., & Fox, D. (2023). *cuRobo: Parallelized collision-free minimum-jerk robot motion generation* (arXiv:2310.17274). arXiv. https://doi.org/10.48550/arXiv.2310.17274

Terver, B., Yang, T.-Y., Ponce, J., Bardes, A., & LeCun, Y. (2026). What drives success in physical planning with joint-embedding predictive world models? *Transactions on Machine Learning Research*. https://doi.org/10.48550/arXiv.2512.24497

Tillet, P., Kung, H. T., & Cox, D. (2019). Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages* (pp. 10–19). ACM. https://doi.org/10.1145/3315508.3329973

Vlahov, B., Gibson, J., Gandhi, M., & Theodorou, E. A. (2024). *MPPI-Generic: A CUDA library for stochastic trajectory optimization* (arXiv:2409.07563). arXiv. https://doi.org/10.48550/arXiv.2409.07563

Williams, G., Aldrich, A., & Theodorou, E. A. (2017). Model predictive path integral control: From theory to parallel computation. *Journal of Guidance, Control, and Dynamics, 40*(2), 344–357. https://doi.org/10.2514/1.G001921

Williams, S., Waterman, A., & Patterson, D. (2009). Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM, 52*(4), 65–76. https://doi.org/10.1145/1498765.1498785

Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association, 22*(158), 209–212. https://doi.org/10.1080/01621459.1927.10502953

Zhou, G., Pan, H., LeCun, Y., & Pinto, L. (2025). DINO-WM: World models on pre-trained visual features enable zero-shot planning. In *Proceedings of the 42nd International Conference on Machine Learning* (Vol. 267, pp. 79115–79135). PMLR. https://proceedings.mlr.press/v267/zhou25t.html

Appendix A. Data dictionary

The solution pack's `data` folder holds five comma-separated tables, produced from the raw benchmark records by `lewm_flash.py prepare`.

Table A1. Columns of the five datasets.

file	column	meaning
calls.csv	config	reference, pytorch_optimised, megakernel, or one of four probes (probe_cached_cem, probe_cached_icem, probe_mk_cem, probe_mk_icem)
	role	system (compared in the evaluation) or probe (isolates one effect for the profile)
	seed, call_index	episode seed (1000 to 1060) and policy call within the episode (1 to 50)
	latency_ms	end-to-end latency of the call, CUDA event timers
episodes.csv	config, seed, episode	system, evaluation seed (0 to 2) and episode index (0 to 99); the same 300 episodes for every system
	success	1 if the T-block reached the goal pose within the 50-call budget, else 0
cost_scaling.csv	execution, candidates	execution method and number of candidates in one scoring call
	p50_ms, n	median latency and number of timed calls
components.csv	component, value, unit, source	component timings (encoders, searches, synchronisation primitives) and planner telemetry (encoder calls per plan, share of a planning call spent in the search)
parity.csv	check	scoring case (problem, number of candidates, sampling scale), encoder check
	pytorch_bf16_rel_max, megakernel_rel_max	maximum relative error against FP32 (relative L2 norm for the encoder row)
	pytorch_bf16_top30, megakernel_top30	number of the FP32 top-30 candidates also in the method's top 30

Appendix B. Solution pack

The solution pack (solution_pack/ in the project repository, also submitted as an archive) contains the datasets, the solution program and its outputs.

Table B1. Contents of the solution pack.

path	content
lewm_flash.py	the solution program: data preparation, preprocessing, compute-profile analysis, statistical evaluation, tables and figures
data/*.csv	the five datasets of Appendix A (460 KB)
outputs/results.json, outputs/tables.md	every statistic in this report, as generated
outputs/figs/*.png	the data figures of this report (Figures 3, 5, 6 and 7)
kernel/ (archive only)	the GPU kernel and its Python runtime, packer and planner class, copied from megakernel/
README.md	how to run

The program needs Python 3.10 or later with numpy and matplotlib, and runs on any computer. Without arguments it runs analyse, which regenerates outputs/ in about five seconds; prepare rebuilds data/ from the raw records inside the repository; and reproduce prints the commands that re-collect the data on a cloud H100.

```
pip install numpy matplotlib
python lewm_flash.py
python lewm_flash.py prepare
python lewm_flash.py reproduce
```

analyse performs, in order: loading and validating the datasets; warm-up removal, call classification and outlier flagging; the stage decomposition, scaling-model fits and per-layer costs; bootstrap intervals, speed-up intervals, Wilson intervals, paired McNemar tests, paired bootstrap intervals and the power calculation; and writes the tables, the JSON results and the figures. Re-collecting the raw data (reproduce --yes) launches the benchmark on rented H100 GPUs through Modal and requires a Modal account, the LeWM checkpoint and the kernel build; it costs roughly $0.20 per seed per system.

Appendix C. Configuration and repeatability

Table C1. Planner configuration of LeWM-MK (identical to the optimised PyTorch planner; the reference uses CEM with 300 candidates in every iteration and no retained elites).

setting	value
candidates per iteration	$\max(\lceil 300 \cdot 1.25^{-i} \rceil, 60)$, rounded up to a multiple of 60: 300, 240, 240, 180, 180, 120, 120, 120, then 60 for 22 iterations
iterations, elites, retained elites	30, top 30, 5 per iteration
rollouts per plan	2,820 (reference: 9,000)
horizon, replanning interval, action block	5, 5, 5
initial sampling scale	1.0
coloured noise, momentum, clipping	not used (reduced success on Push-T in the earlier planner study)

Table C2. Kernel configuration on the H100.

item	value
grid	7 clusters of 16 blocks (one robot); 15 clusters of 8 for batched evaluation
block	256 threads (two groups of 128), 255 registers per thread, 231,584 bytes of shared memory
weights	1,472 blocks of 24 KB in BF16, pre-arranged in the tensor-core layout (964 for the predictor, action encoder and projector; 508 for the encoder)
random numbers	Philox 4×32-10, keyed by plan seed, iteration, robot, candidate and action element
measured primitives	cluster barrier 0.68 µs, grid barrier 1.56 µs, empty kernel launch 14 µs

The latency benchmark was run twice. Table C3 compares the earlier pass with the pass used in this report for the configurations both contain. The PyTorch configurations, which depend on the CPU for every launch, moved by up to 18%; the configurations running the kernel moved by at most 2.2%.

Table C3. Median planning latency in two passes of the benchmark (ms).

configuration	earlier pass	this report	change
reference	526.9	622.4	+18.1%
optimised PyTorch	196.3	182.1	−7.2%
probe: encode once, CEM	391.4	377.0	−3.7%
probe: encode once, iCEM	410.3	410.2	0.0%
probe: kernel search, CEM	81.7	82.1	+0.5%
probe: kernel search, iCEM	58.0	59.3	+2.2%

Appendix D. Compute and cost

All GPU work ran on rented NVIDIA H100 80GB HBM3 instances on Modal, with kernel builds on CPU instances. The project used about $31.50 of GPU time against an approved budget of $35. Most of it (about $21) went on the two latency passes, because the benchmark harness starts a fresh process for each of the 61 seeds, rebuilding the simulator, reloading the model and, for the compiled baseline, recompiling, so that most of a pass's GPU time is setup rather than measurement. The remaining $10.40 covered kernel development, numerical checks, micro-benchmarks and the 300-episode evaluations. A harness that keeps one process across seeds would cut the cost of a latency pass by an order of magnitude.

Appendix E. Code and data availability

All code, raw records, datasets and analysis are in the project repository (lewm-flash, branch antoinenguyen27/lewm-megakernel). The kernel is megakernel/lewm_mk.cu with its packer (pack.py), runtime (runtime.py) and planner class (solver.py). The cloud entry points are in modal/megakernel_app.py, and the benchmark harness is tools/benchmarks/lewm_plan_benchmark.py. The raw per-call records are under results/runs/, and the micro-benchmark and numerical records under results/megakernel/. The solution program and datasets are in solution_pack/, and the report sources in docs/paper/src/. The model is LeWM's released Push-T checkpoint, evaluated with stable-worldmodel's swm/PushT-v1 environment; ThunderKittens is used at commit 67 845f5f.